

AEGIS

A deny-by-default governance control plane for LLM agents on AWS

Partner brief · 10 September 2026

VERIFIED AGAINST

v0.7.0-pilot-rc1

21 / 21

full-portfolio gate, from zero, torn down
to zero residue

*One pack · one account · one region ·
synthetic data*

A reference architecture, not a product. No SLA, no support contract, no certification. Every piece of evidence in it was produced by its author.

Most “AI governance” is a prompt and a logging wrapper

Both of those live inside the thing being governed. An agent that can reason about its instructions can reason around them, and a log written after the fact records what happened rather than preventing it.



Governance in the prompt

The model is asked to comply. Compliance is a behaviour, not a control, and it degrades under adversarial input.



Governance in application code

Better — but the agent chooses which code path to call. The control is downstream of the decision it is meant to constrain.



Governance in front of the model

Authorization is evaluated by AWS before the tool runs. The agent cannot reach the decision point, so it cannot argue with it.

Aegis takes the third position. Cedar authorization, masking proven by signature, per-tenant spend ceilings, a kill switch and a WORM audit ledger sit in front of the model — so the governance cannot be talked out of by the agent it governs.

WHAT IT IS

A configurable control plane, proven end to end on one pack

13

Cedar policies at the gateway
12 forbid · 1 permit

9

CloudFormation stacks
deployed from zero

27

CDK configuration switches
network · KMS · tenancy · retention

4

vertical packs on one
hash-locked core

What it governs

- Who may call which tool, from a verified token — never the request body
- Whether the payload is provably de-identified before any consequential action
- Whether the spend ceiling still allows the call, checked before the spend
- That the record is written before the side effect, and cannot be altered after

How it is configured

- An agent manifest declares the tool targets and the guardrail
- render.py emits gateway targets, Cedar statements and guardrail config
- CDK context switches set network mode, KMS, tenancy, retention, perimeter
- A customer authors their own policies against their own manifest

The four packs share one governed-core artifact pinned by sha256. That is evidence the core ports — not evidence the other three work. Only one has ever been deployed.

One design bet, and everything else follows from it

Put the authorization decision somewhere the agent cannot reach, and make every downstream control depend on a proof rather than a claim.

1 Deny-by-default, forbid-wins

Twelve forbid policies and one permit. Any forbid that matches kills the request. A caller with a missing or malformed claim gets zero tools, not all of them — the failure mode most agent stacks get backwards.

2 Proofs, not booleans

deidentified: true from a caller is never accepted. The proof is a signed sanitized_ref, and a forged reference is refused. Raw case text and the drafted notice never cross workflow state at all.

3 A controller the model does not drive

A deterministic Step Functions machine runs a guard after every substantive step and cannot advance on unverified state. The model is a participant, never the controller.

4 The record is a control, not a log

The audit row is written before the side effect, hash-chained, append-only, with IAM Deny on update and delete — and it carries the policy version that decided the case.

Eight layers, and what each one contributes

- | | |
|--|---|
| 1 Identity & admission | Cognito issues the token carrying groups, tenant and tool entitlement. Tenancy comes from the token, never the request. |
| 2 Authorization | AgentCore Gateway (MCP, CUSTOM_JWT) + the GA Cedar engine in ENFORCE + the interceptor that sets authoritative context. |
| 3 Agent runtime & model | Runtime on the CDK-declared role, MMDSv2 required. Bedrock over PrivateLink. Guardrail ARN pinned by an IAM condition. |
| 4 Deterministic orchestration | Step Functions, ten states, a guard after every substantive step. Human sign-off with the approver re-verified. |
| 5 Governed tools | One Lambda per manifest target, exact-ARN least privilege, private subnets — measured 0 NAT, 0 IGW. |
| 6 Evidence & regulated data | Per-tenant physical isolation. Hash-chained ledger. WORM vault. Customer-managed KMS throughout. |
| 7 Capture & lineage | One account trail, log-file validation on, reconciled against audit rows in both directions. |
| 8 Operations & containment | Kill switch, per-tenant budget, Budgets ceiling. 26 named alarms in 3 severity tiers, each with a runbook entry enforced by CI. |

Full detail — every component, its purpose, what it stops, and whether it is live-verified or IaC-asserted — is in the attached AEGIS-ARCHITECTURE-VERIFIED-2026-09-10.drawio.

Twelve forbid. One permit. That asymmetry is the architecture.

12

forbid

The product. Cedar is forbid-wins: any single match kills the request, no matter how many permits fire.

1

permit

Only the door being unlocked. Grants nothing on its own — every forbid above still wins over it.

What this buys: a caller whose token is missing a claim is granted **zero** tools, not all of them. Optional context fields are presence-guarded, so an absent value fails closed — the forbid fires. The GA engine rejects an unguarded optional access outright, which is why every policy reads “context.input has X && ...”.

Proven live

13 policies reached ACTIVE, twice, in the 2026-09-10 gate — and in that gate deny-by-default reached a newly added tool with no new policy written. The decision is made by AWS, before the tool Lambda is invoked — and it is written to the hash-chained ledger with the policy version that made it (policy_version = cedar-534be1c676bb).

The thirteen policies, grouped by what they defend

A Admission

Is this caller allowed to do anything at all?

`require_tenant`
`require_entitlement`
`require_service_window`

No tenant, no entitlement, or outside the service window → zero tools. Tenancy is derived from the caller's own validated token, so cross-tenant access is impossible by construction.

B De-identification

Nothing consequential touches un-masked data

`mask_before_assess`
`mask_before_draft`
`mask_before_redetermine`
`mask_before_overpayment`

The four consequential actions are forbidden unless the payload is provably de-identified — and the proof is a signature, not a boolean.

C Separation of duties

What the agent may never do

`no_self_commit`
`no_self_fraud_referral`

Forbidden outright and hidden from the advertised tool list. The only path to a commit is the human sign-off gate.

D Purpose, spend, limits

Why, how much, how large

`consent_purpose_before_assess`
`budget_before_draft`
`amount_cap_overpayment`

Purpose limitation; a spend gate set from the live per-tenant meter, refusing before the spend; and a dollar cap above which a case needs elevated authorization.

Honest limit: these thirteen are a worked example, not a reusable library — they name benefits-specific tool actions. A customer authors their own against their own manifest, which is the real platform capability.

Masking is proven, not asserted

What most systems do

The caller passes `deidentified: true` and the platform believes it. The control is a self-report by the party it constrains.

What Aegis does

A cryptographically signed `sanitized_ref`, issued only by `mask_pii`. A forged or tampered reference is refused by the guard.

And the raw application text — and the drafted notice — never cross workflow state at all. Only opaque references do. The strict PII canary found zero leaks across Logs, X-Ray, DLQ and Step Functions history.

Why this is also the answer to “what if someone bypasses the gateway?”

We tested a control (RT-4) that restricted the runtime to gateway-only invocation. It worked — and because the gateway sits downstream of the runtime in this topology, it left the agent with no permitted invoker and failed four gate checks. It is now opt-in and off by default, so R4-2 is OPEN and we say so. The bypass is mitigated where it actually bites: the four `mask_before_*` forbids refuse the consequential actions without the signature, whoever calls. RT-4 would have closed a path; these close the action.

21 of 21, from zero, then torn down to nothing

PROVEN LIVE

**Deploy from zero**

rc=0 in 722s · nine stacks

**Cedar ENFORCE**

13 policies ACTIVE, twice

**Guardrail on every call**

runtime_rows=20 · assessed=20

**Multi-tenant isolation**audit routing 12/12 · transparency 13/13
per tenant**Kill switch**

engaged, blocked, disengaged · 105s

**Per-tenant budget**

refused before the spend · 373s

**Unified lineage**

orphans=0 · 11 invokes / 11 rows

**Full case driven**

LIN-0E6D3D through all ten states

**Policy provenance**

policy_version in a live ledger row

**Teardown**

stack, AgentCore AND connector residue

Scope of that claim, exactly: one pack, one AWS account, one region, two tenants, synthetic data. Run ben-fpg, 10 September 2026. Evidence: [evidence/FULL-PORTFOLIO-GATE-benefits_runtime_agent.json](#)

THE ONE THING THAT CHANGED SINCE 9 SEPTEMBER

A governed tool now reaches a real OAuth2 system of record. It is ours.

What is now proven

- `verify_source` is a Gateway target and holds NO client secret
- The outbound token is minted by the AgentCore Identity vault (M2M)
- The system of record verifies RS256 against the issuer's live JWKS
- Cedar denied an outsider — naming a specific policy, not refusing everyone

What it still is not

- The system of record is REAL — and it is OURS. Not a customer's.
- `connect_system_of_record` remains stubbed in the manifest, on purpose
- No customer IdP, network boundary, token lifetime or change control
- Six from-zero live runs to get this far: treat it as engineering work

Why this slide is here. Nothing in the previous edition of this deck claimed a governed connector, because until 10 September it had never once run to completion. Not one of the six failures was where it appeared to be: authentication, then the gateway authorizer, then the system of record itself — each hidden behind an error written to `/dev/null`. The gate that now proves it is stricter than the gate that used to pass it.

The two controls that stop it recurring

`CONN_deploy` used to pass on the connector's artifacts merely existing, and did so twice over a system of record that trusted nothing. It now also requires the deploy to have exited 0 and the outbound leg to have been exercised — a rule validated by replay against those two failed runs before it shipped. And CI now fails when a new silenced error path appears.

It took seven attempts. All six failures are published.

The run immediately before the pass reached the checks and failed four of them. It sits in the same commit as the pass. If you read one thing to decide whether this evidence is honest, read those two files side by side.

L60 Line-ending noise that was cosmetic — and which I first diagnosed as digest corruption and was wrong. Tested before publishing; the claim was false and the commit message was rewritten.

L64 Three consecutive wrong diagnoses of one `ConflictException`. Real cause: AgentCore policy names are unique per account and region, not per policy engine. The repo's own code comment had said so all along.

L61 The CloudTrail five-trails-per-region quota, which AWS marks not adjustable. A service limit, not a defect. Still open.

L65 / L66 RT-4 — a control that worked and made the agent unreachable. Reverted to opt-in; R4-2 reopened.

Seventy findings are recorded this way, including every one where I was wrong — and the connector on the next slide took six more from-zero runs after this one. That register is the strongest credibility asset in the repository.

Read this list before any customer conversation

NOT PROVEN

All evidence is author-produced

One AWS account. No third party has run it. No amount of further self-testing closes this.

No CUSTOMER system of record

A governed OAuth2 connector is now live and proven — but the system of record is ours. `connect_system_of_record` stays stubbed. Every live run governed synthetic data.

No account-wide prevention

No AWS Organization, therefore no SCP. A direct Bedrock call is detected, not prevented.

R4-2 is open, deliberately

Gateway-only runtime invocation was tested live and reverted. Mitigated at the gateway, not closed.

Three of four packs never deployed

Same core, propagated fixes, unit tests — zero from-zero deployments. Housing wires 3 of 17 controls.

COMPLIANCE lock proven at 1 day

The mechanism is proven; the seven-year duration is a parameter, not evidence.

No pen test, DR, SLO or ATO

None performed. The platform carries no authorization of any kind.

488 checkov findings baselined

Reviewed and gated so new findings fail CI — but the residual is real.

This list is longer than the proven list, and it is in the deck on purpose. NOT-CLAIMS.md governs every other page in the repository.

We said three alarms. It was twenty-two.

What we claimed

- "Three alarms. Observability is the thinnest part of the platform."
- Counted `cw.Alarm`(CALL SITES in the source - two are inside loops
- Measured the generating code instead of the deployed artifact
- Same defect as two entries already in our own register (L60, L67)

What is actually deployed

- 26 named alarms in 3 severity tiers - P1 page, P2 ticket, P3 advisory
- Latency alarmed, not merely charted. Approval backlog: 2h, was 24h
- Every alarm has a runbook entry, enforced by a test in CI
- Still open: per-tenant dashboards, canary, proof a tier fires live, DR

Why this slide is in the deck at all. A number in our own material was wrong by a factor of seven, in the direction that flattered nobody. How a team handles its own bad number is worth more signal than the number was — and the fix is not "measure more carefully next time", it is a script that reads the CloudFormation template and fails CI when these figures drift from what the brief says.

The control that stops it recurring

`tools/observability_census.py` reads the synthesized template and counts what actually deploys. `--assert-baseline` fails if the numbers drift from what the partner documents quote, so the brief and the infrastructure cannot disagree silently. Ask your SA to run it.

One thing, and it is cheap: run it yourself

“All of the evidence is ours, which means none of it counts. We want your SA to run the verification protocol in a clean account. It is scripted, it takes about half a day, and it either reproduces our results or it does not.”

1	Clone the tag	10 min	v0.7.0-pilot-rc1. Confirm tag == main and the RELEASE file inside the tag names the tag.
2	Run the offline suite	20 min	510 pass, 4 skip (514 collected). Confirm the CI security gates are blocking, not report-only.
3	Read a pass and a failure	20 min	A pass and the failure before it, side by side, in the same commit. Cheapest high-signal step in the list.
4	Deploy from zero	~2 hr	Into a clean account you control, following docs/INDEPENDENT-VERIFICATION.md.
5	Run the gate yourself	~1.5 hr	Record the result — including if it fails. That is the deliverable either way.
6	Try to break it	~1 hr	Forge a sanitized_ref. Call a tool with no entitlement. Call finalize_determination directly.
7	Tear down	30 min	Confirm zero stack and zero AgentCore residue in your account.

One prerequisite we owe you: the account needs CloudTrail headroom. The limit is five trails per region and AWS marks it not adjustable. Each deployed pack needs two.

Verify, then extend, then a customer — in that order



Verify

Weeks 1–2

Partner SA reproduces the gate in a clean account, including the break-it step.

Gate: it reproduces, or we learn why not.



Extend

Weeks 3–8

The same connector proof against a real EXTERNAL system of record — ours is proven, a customer's is not. Live re-gate of a second pack. Observability uplift. Organization + SCP if you can supply an OU.

Gate: a system of record we do not own is governed.



Pilot

Quarter 2

Named customer. Supervised pilot on de-identified data. Customer's own security engineering, testing and authorization.

Gate: the customer's AO accepts the compensating-control statement.



Production

Later

Not in scope for this conversation. Pen test, DR, SLO, IdP round-trip, ATO, formal authorization.

—

We are deliberately NOT asking you to find a customer yet. That would be asking you to put your name on synthetic, author-produced evidence.

Where this actually stands

What it is

A credible governance control plane at reference-architecture maturity, with an unusually well-documented gap list — and further along than most teams will have built themselves.

Good for: architecture workshops, a CISO/AO conversation, a supervised pilot on synthetic or de-identified data.

What it is not

Not a product. Not an AWS service. Not certified, authorized or ATO'd. Not ready for real regulated data. Not independently verified by anyone.

Say this first. It costs nothing, and it buys you the rest of the conversation.

The one thing we need from you

Half a day of an SA's time, in an account we do not control, to find out whether any of this survives contact with someone who is not us.